Image Classification on Garbage Dataset Using CNNs and Transfer Learning

---

Student Name: Pratik Dhimal

Student ID: 2407779

Tutor: Ms. Sunita Parajuli

Submission Date: May 9, 2026

Part II - Computer Vision Task

GitHub Link:

https://ln.run/ISIEA

Contents

1 Introduction

Sorting waste by hand is a slog, but it's exactly the kind of thing we can automate. If we can get computer vision to sort garbage accurately, we can make recycling way more efficient and cut down on mistakes. For this project, I used the Garbage Classification v2 dataset from Kaggle to see how three different neural network setups stacked up.

I wanted to find out:

1. How does a basic CNN handle this out of the box?
2. Does adding layers and normalization make a real difference?
3. Is transfer learning actually as big an advantage as people say?

2 Dataset

2.1 Overview

The dataset has 12,259 images split into 10 categories, from batteries and glass to shoes and general trash. The images were gathered through crowdsourcing and web scraping, then labeled manually.

2.2 Data Distribution

Class	**Count**	**%**
Clothes	1,892	15.43
Glass	1,736	14.16
Plastic	1,597	13.02
Cardboard	1,411	11.51
Paper	1,336	10.90
Shoes	1,449	11.82
Metal	930	7.59
Battery	756	6.17
Biological	699	5.70
Trash	453	3.69

Table 1: Class distribution. Trash is the rarest category at 3.69%.

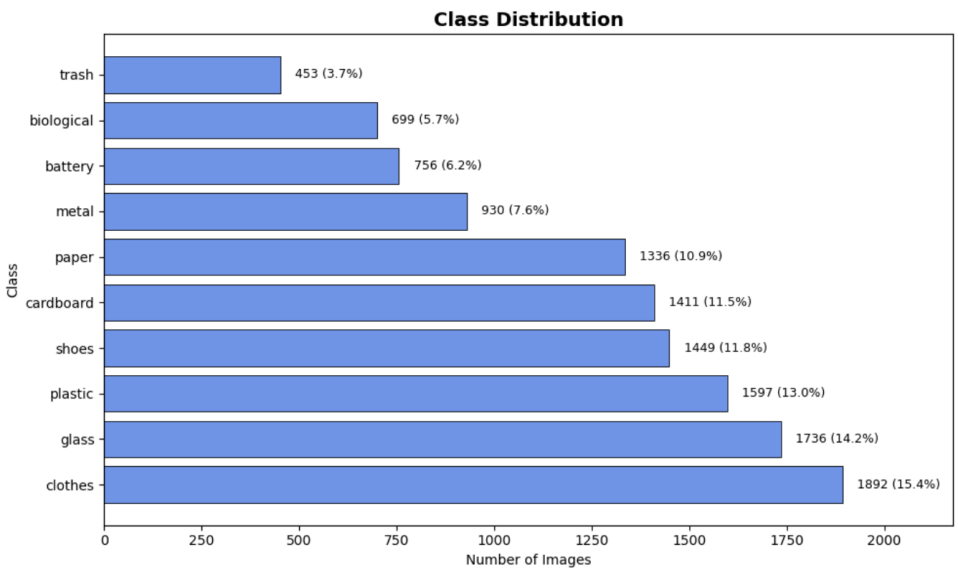

Figure 1: Class distribution histogram

2.3 Preprocessing

I resized everything to 224x224 pixels and normalized the values using ImageNet stats. During training, I added some variety with data augmentation—random rotations, shifts, and flips—to help the model generalize.

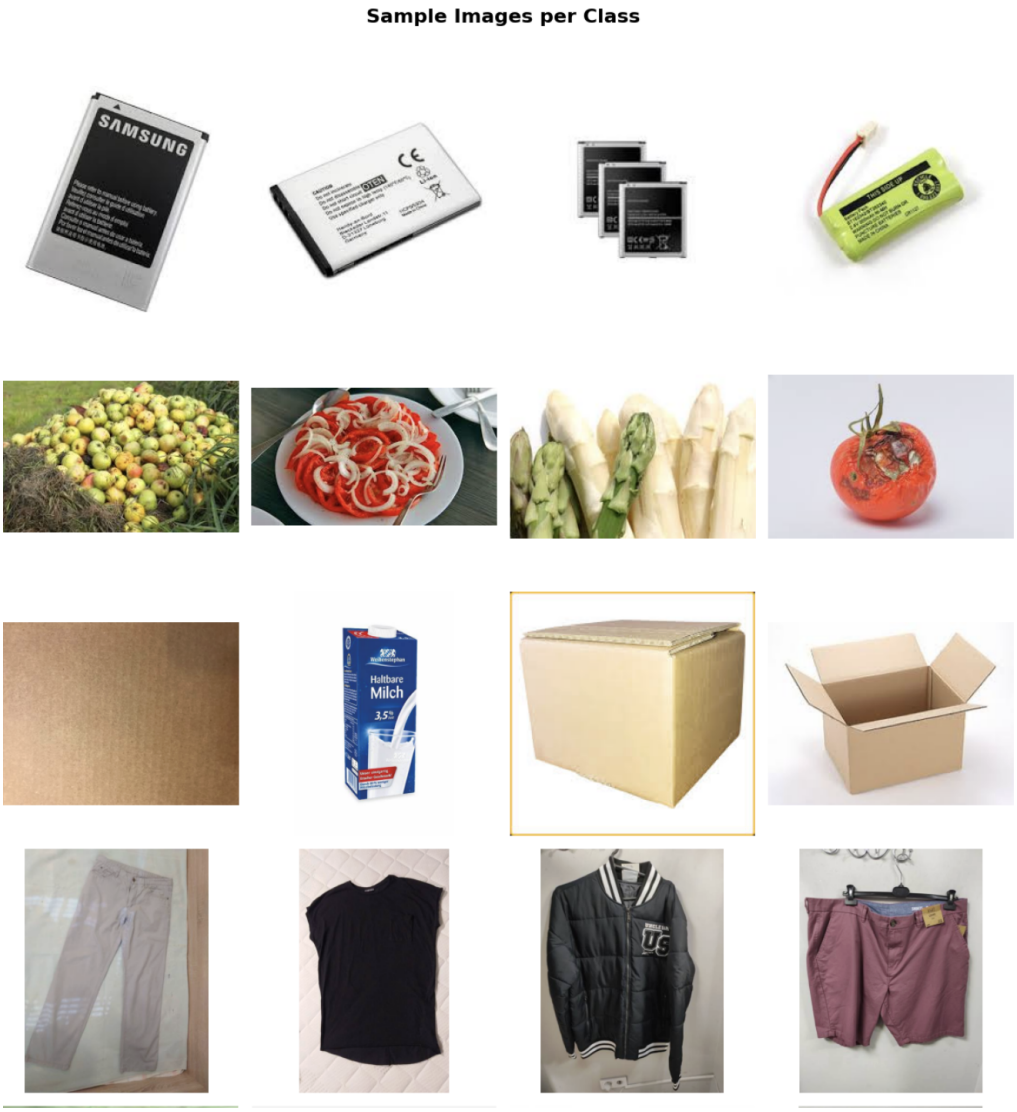

Figure 2: Sample images from each class

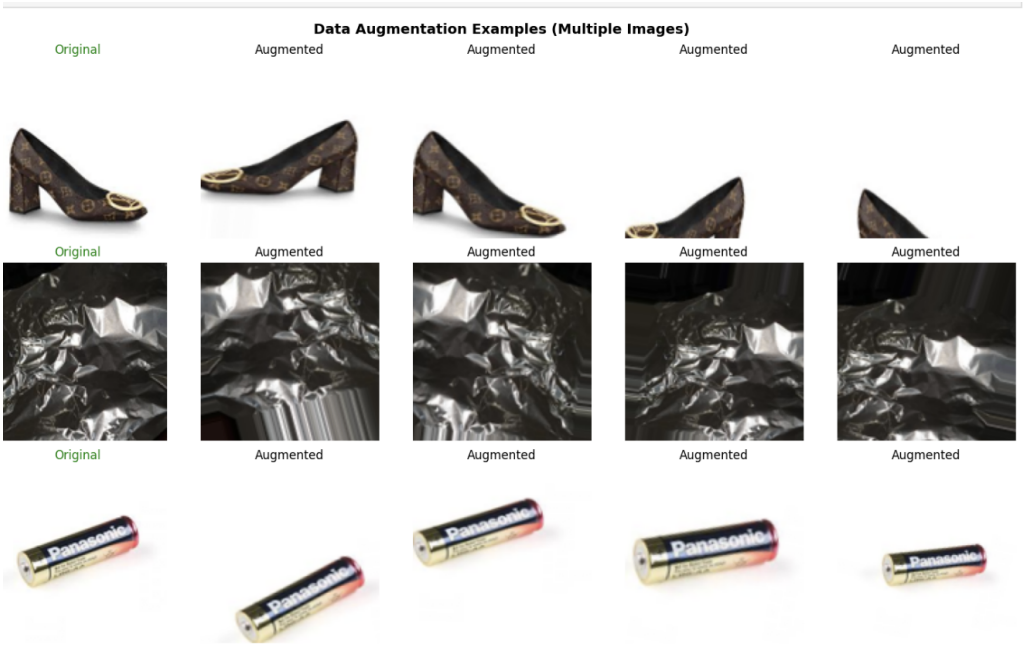

Figure 3: Examples of how data augmentation affects the images.

I used a 70/15/15 split for training, validation, and testing, using stratified sampling to make sure each group had a fair representation of the different classes.

3 Methodology

3.1 Model 1: Baseline CNN

Architecture:

- 3 conv blocks (filters increasing 32 to 128)
- 3×3 kernels and 2×2 max pooling
- Fully connected layers: 512, then 128, then 10 (the classes)
- 5.4M parameters total

Training: Adam optimizer (0.001), batch size 32, up to 25 epochs with early stopping.

3.2 Model 2: Deeper CNN with Regularization

Architecture:

- 6 conv layers (filters: 24, 48, 96)
- Batch Normalization after every conv layer
- Dropout (0.1) on the final layers to prevent overfitting
- 7.8M parameters total

Training: Same optimizer and batch size, but ran for up to 35 epochs.

3.3 Model 3: Transfer Learning (MobileNetV2)

Architecture:

- Started with a pre-trained MobileNetV2 backbone (ImageNet)
- Phase 1: Froze the backbone and just trained the new classification head for 15 epochs.
- Phase 2: Unfroze the last 30 layers and fine-tuned with a very low learning rate (0.00001) for 20 more epochs.

4 Results

4.1 Overall Comparison

Model	Accuracy	Precision	Recall	F1
Baseline CNN	72.59%	0.7284	0.7259	0.7249
Deeper CNN	79.70%	0.7984	0.7970	0.7944
MobileNetV2 TL	93.38%	0.9350	0.9338	0.9339

Table 2: Final performance summary across all models.

4.2 Experiment 1: Baseline vs. Deeper CNN

Adding depth and Batch Normalization bumped the accuracy up by about 7 points (from 72.59% to 79.70%). Both models eventually started to overfit, but the deeper model was much more stable during training.

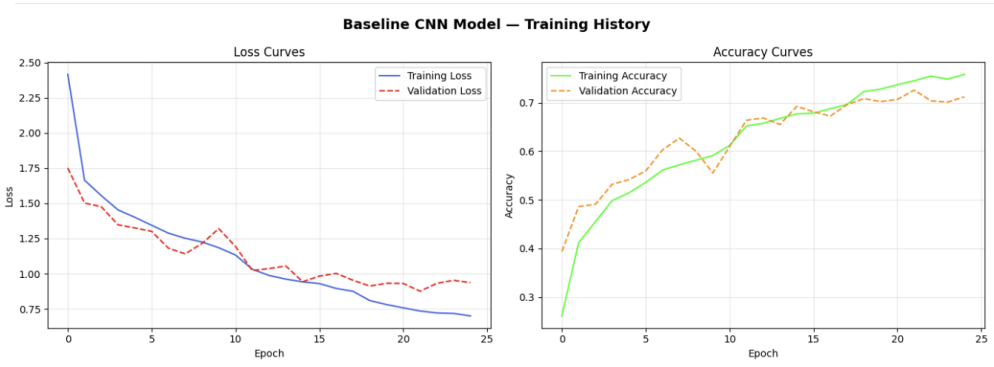

Figure 4: Baseline training curves.

Figure 5: Deeper model training curves.

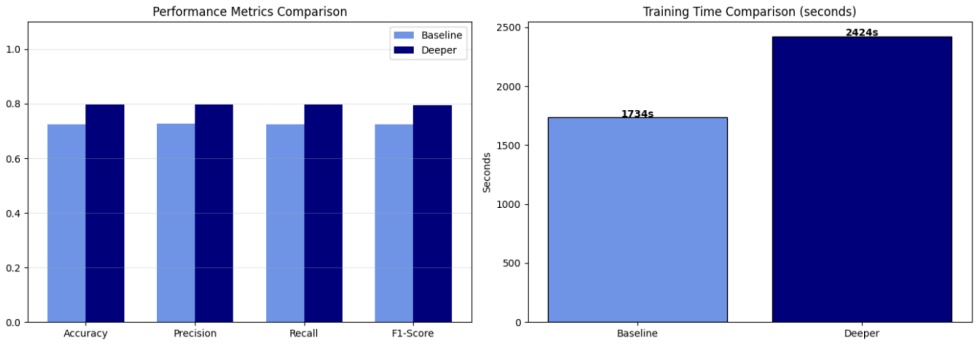

Figure 6: Validation loss comparison.

4.3 Experiment 2: Optimizer Analysis

Optimizer	Accuracy	Training Time (s)
Adam	79.70%	2,423.7
SGD	71.57%	1,958.0

Table 3: Adam vs. SGD comparison.

Adam outperformed SGD by over 8%. The adaptive learning rate clearly works better for this kind of image data.

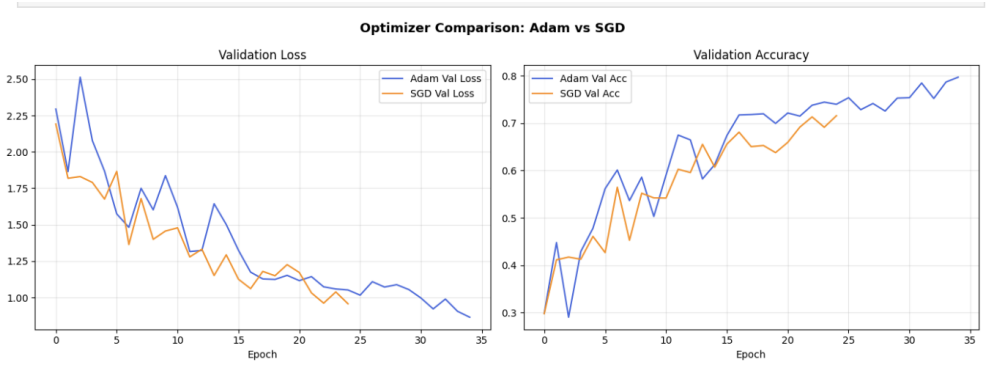

Figure 7: Training progress: Adam vs. SGD.

4.4 Experiment 3: Ablation Study

Configuration	Accuracy
Deeper CNN (with BN)	79.70%
Deeper CNN (without BN)	72.00%

Table 4: What happens when you remove Batch Normalization.

When I took Batch Normalization out, accuracy fell back down to 72%. It's pretty clear that normalization is what kept the deeper model from falling apart.

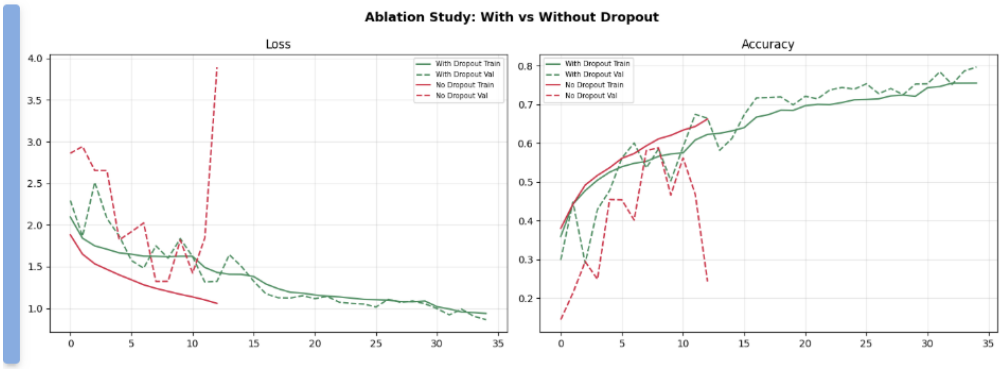

Figure 8: Validation loss after removing Batch Normalization.

4.5 Experiment 4: Transfer Learning

MobileNetV2 was on another level, hitting 93.38% accuracy.

Phase 1 (Feature Extraction):

- It started at 72.42% accuracy in the first epoch and hit 88.60% in under four minutes.

Phase 2 (Fine-Tuning):

- Fine-tuning the top layers brought it up to the final 93.38% mark.

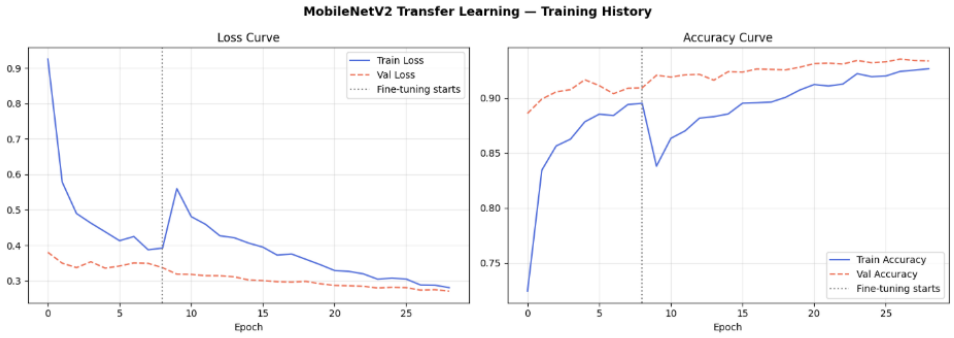

Figure 9: MobileNetV2: Phase 1 history.

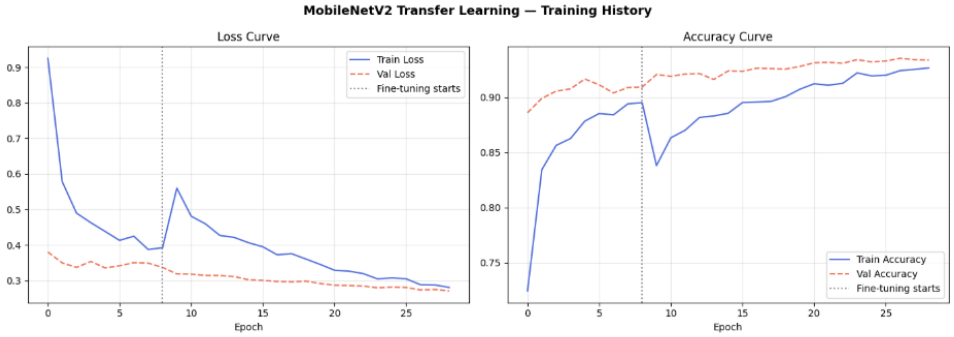

Figure 10: MobileNetV2: Phase 2 history.

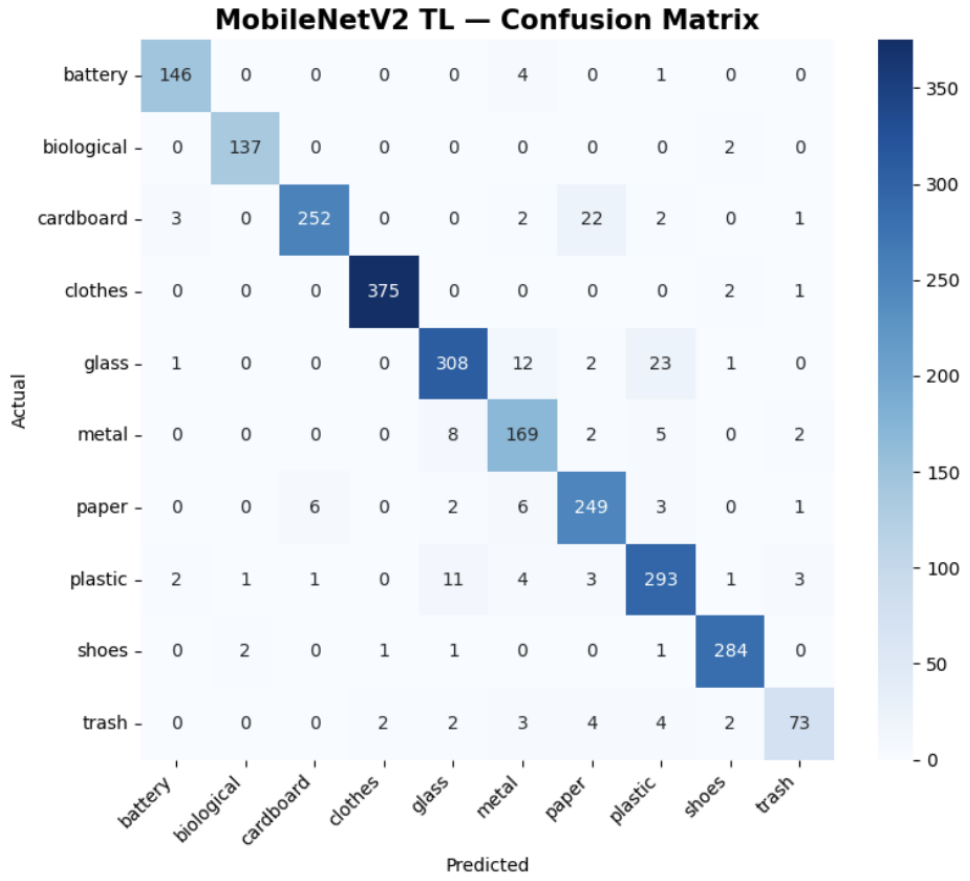

Figure 11: Full training history for MobileNetV2.

4.6 Model Comparison

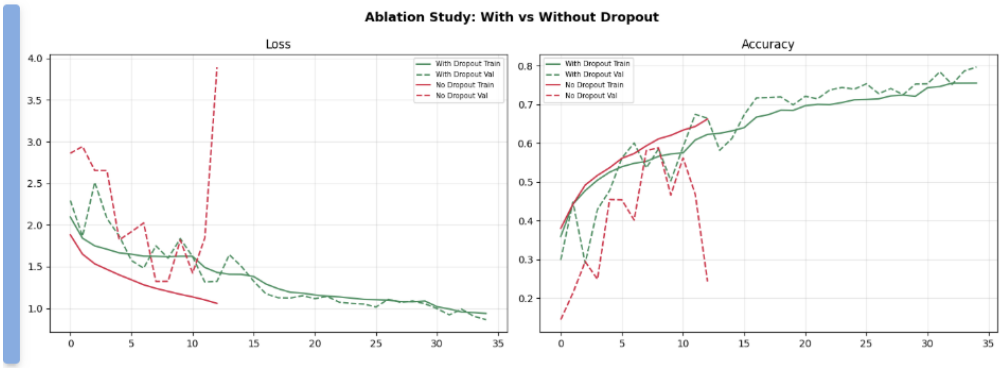

Figure 12: Validation loss for all three models.

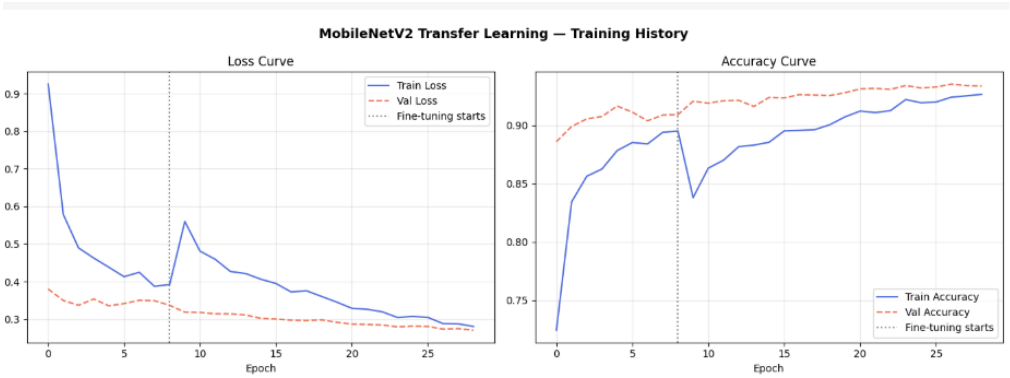

Figure 13: Validation accuracy for all three models.

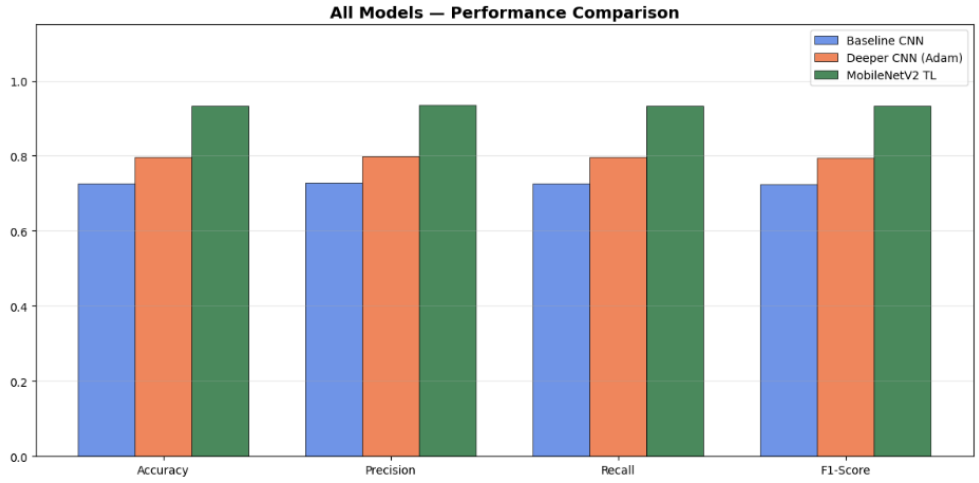

Figure 14: Comparing accuracy, precision, recall, and F1.

4.7 Computational Efficiency

Model	Parameters	Training Time (s)
Baseline CNN	5.4M	1,734
Deeper CNN	7.8M	2,424
MobileNetV2 Phase 1	0.5M trainable	231
MobileNetV2 Full	3.2M trainable	2,151

Table 5: Comparison of parameters and training time.

5 Discussion

Transfer learning was over 20% more accurate than my baseline CNN. The pre-trained features from ImageNet are just a massive head start. In fact, the first phase of MobileNetV2 training hit 88.60% in just 231 seconds—way faster than any of the custom models.

I tried making the custom models deeper and adding things like Dropout and Batch Normalization, but it only got me so far. The deeper CNN peaked at 79.70%. It really feels like for a specific task like garbage sorting, the quality of your pre-trained features matters more than how many layers you throw at it from scratch.

6 Conclusion

If we are building a garbage classifier, it can be said than never start from zero as it will increase computational powere with no result

1. MobileNetV2 hit 93.38% accuracy, whereas my best custom model barely touched 80%.
2. ImageNet features translate surprisingly well to garbage images.
3. Fine-tuning gave me a solid 5% boost over just training the classification head.
4. MobileNetV2's first phase was 7.5x faster to train than the baseline.
5. Batch Normalization and Adam are good to have, but they won't save a model that doesn't have good feature representations.

What's next?

- I want to see if EfficientNet or Vision Transformers can do even better.
- I'd like train the model in a good computational power computer to see the difference as it took a lot of time for me